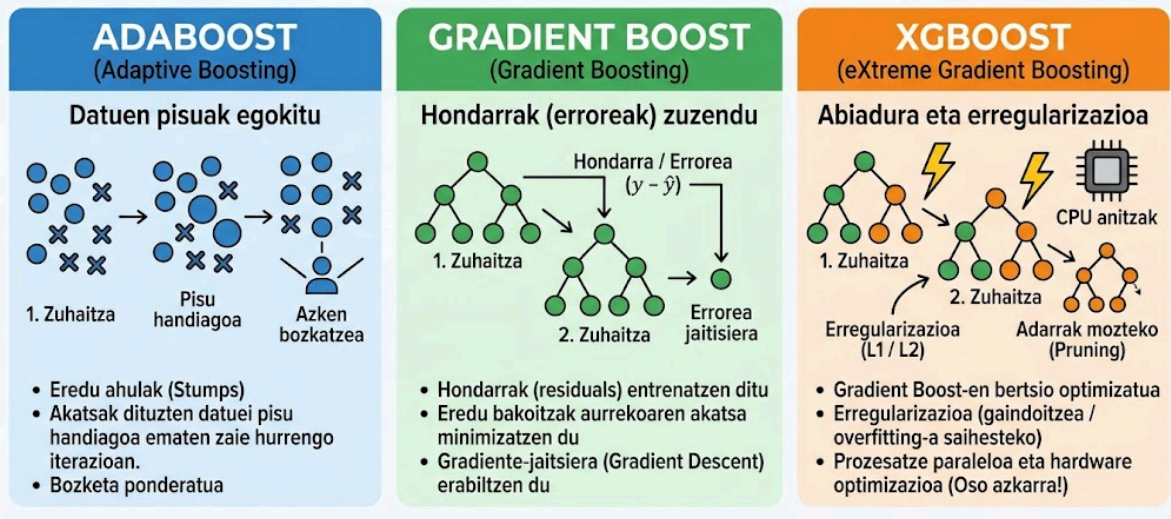


BOOSTING ALGORITMOAK

Sekuentzialki ikasten duten ereduen multzoak



AdaBoost

Demagun **4 datu-puntu** ditugula, balio bakarreko ezaugarri batekin (x), eta bi klasetan sailkatu nahi ditugu (+1 / -1):

Puntua	x	Klase erreala (y)
P1	1	+1
P2	2	+1
P3	3	-1
P4	4	+1

Ikusten duzunez, ez dago muga bakar bat ($x < t$) P guztiak ongi bereiziko dituen — horregatik behar dugu **zurtoin bat baino gehiago**, bakoitza aurrekoaren hutsunea betetzeko.

1. urratsa — Hasierako pisuak

Hasieran, puntu guztiek garrantzi bera dute:

$$w_i = 1/N = 1/4 = 0.25 \text{ bakoitzarentzat}$$

2. urratsa — Lehen zurtoina (ikasle ahula) entrenatu

Zurtoin sinpleena hartzen dugu: "**x < 2.5 bada, +1; bestela, -1**"

Puntua	Aurreikuspena	Zuzena?
P1	+1	✓
P2	+1	✓
P3	-1	✓
P4	-1	✗ (benetakoa +1 zen)

3. urratsa — Errorea kalkulatu

Oker dagoen punturen pisuen batura zatituta guztien batuaz:

$$\varepsilon_1 = \frac{w_{P4}}{\sum w_i} = \frac{0.25}{1.0} = 0.25$$

4. urratsa — Alpha kalkulatu (zurtoin honen "hitza")

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_1}{\varepsilon_1} \right) = \frac{1}{2} \ln(3) \approx 0.55$$

Zenbat eta errorea txikiagoa, orduan eta alpha handiagoa → zurtoin honek botoan pisu gehiago izango du.

5. urratsa — Pisuak eguneratu eta normalizatu

Erregela erraza: **ongi sailkatutakoei pisua jaitsi, gaizki sailkatutakoari igo.**

- Ongi sailkatuak (P1, P2, P3): $w \leftarrow w \cdot e^{-\alpha_1} = 0.25 \times 0.58 \approx 0.144$
- Gaizki sailkatua (P4): $w \leftarrow w \cdot e^{+\alpha_1} = 0.25 \times 1.73 \approx 0.43$

Normalizatu (batura 1 izan dadin):

Puntua	Pisu berria
P1	0.17
P2	0.17
P3	0.17

P4	0.50 ← orain honi ematen zaio arreta gehien
-----------	--

6. urratsa — Bigarren zurtoina entrenatu

Algoritmoak orain **P4 ongi sailkatzea** saritzen du gehien, pisu handiena baitu. Zurtoin berri hau hautatzen da: "**x < 3.5 bada, -1; bestela, +1**"

Puntua	Pisua	Aurreikuspena	Zuzena?
P1	0.17	-1	✗
P2	0.17	-1	✗
P3	0.17	-1	✓
P4	0.50	+1	✓

P1 eta P2 oker daude orain, baina pisu txikiarekin — eta P4, garrantzitsuena, ondo dago.

7. urratsa — Azken erabakia: botazio haztatua

Zurtoin bakoitzak bere alpha-ren arabera botoa ematen du, eta batura hartzen da (zeinua: + edo -):

Adibidez, P2-rentzat:

$$F(P2) = \text{sign}(0.55 \times (+1) + 0.35 \times (-1)) = \text{sign}(0.20) = +1 \quad \checkmark$$

Laburpena

- **Hasieran** puntu guztiak berdinak dira.
- **Zurtoin bakoitzak** aurrekoak huts egindakoei arreta handiagoa jartzen die (pisuen bidez).
- **Alpha** handiagoa da zurtoin onenentzat → gehiago "entzuten" zaie azken botoan.
- Adibide honetan 2 zurtoinekin ia dena konpontzen dugu; **praktikan 50-100 zurtoin inguru** erabiltzen dira, eta pixkanaka ia errore guztiak desagertzen dira.

Gradient Boosting

Oraingo honetan **erregresio-adibide** bat erabiliko dugu (balio zenbakizko bat aurreikustea), Gradient Boosting-en kontzeptua argiago ikusteko: **hondakinak (erroreak) pixkanaka doitzea**.

Demagun etxe baten **tamaina (x)** ezagututa, **prezioa (y, milakotan)** aurreikusi nahi dugula:

Puntua	x (tamaina)	y erreala (prezioa)
P1	1	30
P2	2	32
P3	3	48
P4	4	50

1. urratsa — Hasierako eredua (konstantea)

Gradient Boosting-ek **batez bestekoarekin** hasten da, hasierako aurreikuspen gisa:

$$F_0(x) = \text{batezbestekoa} = \frac{30+32+48+50}{4} = 40$$

Puntu guztientzat 40 aurreikusten dugu hasieran — oso zehaztasun gutxikoa, jakina.

2. urratsa — Hondakinak kalkulatu (errorea)

$$r_i = y_i - F_0(x_i)$$

Puntua	y erreala	F ₀ aurreikuspena	Hondakina (r _i)
P1	30	40	-10
P2	32	40	-8
P3	48	40	+8
P4	50	40	+10

Hemen dago gakoa: hurrengo zuhaitzak **ez du y aurreikusiko, hondakin hauek aurreikasiko ditu**.

3. urratsa — Lehen zuhaitz ahula (h₁) hondakinei doitu

Zuhaitz sinple bat entrenatzen dugu, adibidez zatiketa **x < 2.5**:

- Ezkerra (P1, P2): hondakinen batez bestekoa = $(-10-8)/2 = -9$
- Eskuina (P3, P4): hondakinen batez bestekoa = $(8+10)/2 = +9$

4. urratsa — Eredua eguneratu (ikasketa-tasarekin, v)

Zuzenean ez dugu zuhaitz osoa gehitzen — **ikasketa-tasa txiki batekin** (v) eskalatzen dugu, urrats txikiak emateko eta ez gainegokitzeko. Demagun $v = 0.5$:

$$F_1(x) = F_0(x) + v \cdot h_1(x)$$

Puntua	F_0	$v \cdot h_1$	F_1 (berria)
P1	40	$0.5 \times (-9) = -4.5$	35.5
P2	40	-4.5	35.5
P3	40	+4.5	44.5
P4	40	+4.5	44.5

Aurreikuspenak benetako balioetara hurbildu dira pixka bat (35.5 vs 30-32, 44.5 vs 48-50).

5. urratsa — Hondakin berriak kalkulatu

$$r_2 = y - F_1$$

Puntua	y	F_1	Hondakin berria
P1	30	35.5	-5.5
P2	32	35.5	-3.5
P3	48	44.5	+3.5
P4	50	44.5	+5.5

Ikusi: hondakinak **txikiagoak** dira orain ($\pm 10 \rightarrow \pm 5.5$). Hor dago Gradient Boosting-en funtsa.

6. urratsa — Bigarren zuhaitza (h_2) eta eguneraketa berria

Berriz zatitzen dugu $x < 2.5$: ezkerra = $(-5.5-3.5)/2 = -4.5$; eskuina = $(3.5+5.5)/2 = +4.5$

$$F_2(x) = F_1(x) + v \cdot h_2(x)$$

Puntua	F_1	$v \cdot h_2$	F_2	Hondakin berria
--------	-------	---------------	-------	-----------------

P1	35.5	-2.25	33.25	-3.25
P2	35.5	-2.25	33.25	-1.25
P3	44.5	+2.25	46.75	+1.25
P4	44.5	+2.25	46.75	+3.25

Hondakinak are txikiagoak: $\pm 5.5 \rightarrow \pm 3.25$. **Prozesua errepikatuz joaten da**, zuhaitz gehiago gehituz, errorea gero eta txikiagoa izan arte.

7. urratsa — Azken eredua

Zuhaitz bakoitza aurrekoaren hondakinei doitzen zaie, eta guztiak batuta (ikasketa-tasarekin eskalatuta) azken iragarpena lortzen da.

Laburpena

- **F_0** : hasierako aurreikuspen soila (batez bestekoa).
- **Zuhaitz bakoitzak** aurreko hondakinak (benetakoaren eta aurreikuspenaren arteko aldea) iragartzen ditu, ez jatorrizko helburua.
- **v (ikasketa-tasa)**: urrats txikiak ematen ditu, gainegokitzea saihesteko (praktikan 0.01–0.3 artekoa izan ohi da, nik adibide honetan 0.5 erabili dut argiago ikusteko).
- **Iterazio gehiagorekin** (praktikan 100-500 zuhaitz), hondakinak ia 0 izatera iristen dira.

AdaBoost-ekin alderatuta: AdaBoost-ek **lagin-pisuak** aldatzen ditu (zein punturi jarri arreta gehiago); Gradient Boosting-ek, ordea, **hondakinak zuzenean** iragartzen ditu zuhaitz berriekin.

Nahi baduzu, diagrama batekin ere ilustra dezaket hondakinen txikitze hau, edo Python kode txiki batekin kalkulu hauek simulatu.

orain azaldu XGBoost. Adibide bat jarri eta azaldu pausuz pausu modu errezean ulertzeko

XGBoost

Jarraituko dugu **etxe-prezioen adibide berarekin**, Gradient Boosting-ekin konparatu ahal izateko. XGBoost-ek funtsean gauza bera egiten du (hondakinak zuzentzen ditu zuhaitz berriekin), baina **zatiketa hoberena aukeratzeko eta gainegokitzea saihesteko** tresna matematiko propioak ditu: *antzekotasun-puntuazioa*, *gain* eta *erregularizazioa* (λ , γ).

Puntua	x (tamaina)	y erreala (prezioa)
P1	1	30
P2	2	32
P3	3	48
P4	4	50

1. urratsa — Hasierako eredua

Gradient Boosting-en bezala, balio konstante batekin hasten gara:

$$F_0(x) = 40$$

2. urratsa — Hondakinak kalkulatu

Puntua	y	F_0	Hondakina
P1	30	40	-10
P2	32	40	-8
P3	48	40	+8
P4	50	40	+10

3. urratsa — Antzekotasun-puntuazioa (Similarity Score) kalkulatu

Hau da XGBoost-en berezitasun nagusia. Formula:

$$S = \frac{(\sum \text{hondakinak})^2}{n + \lambda}$$

non λ (**lambda**) erregularizazio-parametroa den (zenbat eta handiagoa, orduan eta kontserbadoreagoa eredua). Demagun $\lambda = 1$.

Sustraiaren puntuazioa (4 puntu guztiak batera):

$$S_{root} = \frac{(-10-8+8+10)^2}{4+1} = \frac{0}{5} = 0$$

4. urratsa — Zatiketa-kandidatuak probatu eta Gain konparatu

Kandidatua A: $x < 2.5$

- Ezkerra (P1,P2): $S_{left} = \frac{(-10-8)^2}{2+1} = \frac{324}{3} = 108$
- Eskuina (P3,P4): $S_{right} = \frac{(8+10)^2}{2+1} = \frac{324}{3} = 108$

$$Gain_A = S_{left} + S_{right} - S_{root} = 108 + 108 - 0 = 216$$

Kandidatua B: $x < 1.5$

- Ezkerra (P1): $S = \frac{(-10)^2}{1+1} = 50$
- Eskuina (P2,P3,P4): $S = \frac{(10)^2}{3+1} = 25$

$$Gain_B = 50 + 25 - 0 = 75$$

$Gain_A (216) > Gain_B (75) \rightarrow x < 2.5$ hautatzen da zatiketarik onena bezala.

5. urratsa — Hostoen irteera-balioak kalkulatu

$$Output = \frac{\Sigma_{hondakinak}}{n+\lambda}$$

- Ezkerreko hostoa (P1,P2): $-18/3 = -6$
- Eskuineko hostoa (P3,P4): $18/3 = +6$

6. urratsa — Inaustea (Pruning) γ -rekin

Zatiketa bakoitza mantendu edo kendu erabakitzeko: $Gain - \gamma$

Demagun $\gamma = 100$:

$$216 - 100 = 116 > 0 \rightarrow \text{zatiketa mantentzen da}$$

Gain γ baino txikiagoa izan balitz (adibidez $\gamma=250$), zatiketa **kenduko** litzateke, gehiegizko konplexutasuna saihesteko.

7. urratsa — Eredua eguneratu (ikasketa-tasarekin, η)

XGBoost-en ikasketa-tasari **eta** (η) deitzen zaio normalean. Demagun $\eta = 0.3$:

$$F_1(x) = F_0(x) + \eta \cdot \text{Output}(x)$$

Puntua	F_0	$\eta \cdot \text{Output}$	F_1
P1	40	$0.3 \times (-6) = -1.8$	38.2
P2	40	-1.8	38.2
P3	40	$0.3 \times (+6) = +1.8$	41.8
P4	40	+1.8	41.8

8. urratsa — Hondakin berriak eta bigarren zuhaitza

Puntua	y	F_1	Hondakin berria
P1	30	38.2	-8.2
P2	32	38.2	-6.2
P3	48	41.8	+6.2
P4	50	41.8	+8.2

Prozesu bera errepikatzen da: antzekotasun-puntuazioa $\rightarrow$ zatiketa hoberena (Gain) $\rightarrow$ inaustea (γ) $\rightarrow$ irteera-balioak $\rightarrow$ eguneraketa (η). Zuhaitz bakoitzak hondakinak txikiagotzen ditu.

9. urratsa — Azken eredua

$$F(x) = F_0(x) + \eta \cdot h_1(x) + \eta \cdot h_2(x) + \dots + \eta \cdot h_M(x)$$

Laburpena: zergatik da berezia XGBoost?

Kontzeptua	Zertarako balio du
Antzekotasun-puntuazioa	Zatiketa bakoitzaren kalitatea neurtzen du, bariantzaren ordez
Gain	Zatiketa onena hautatzeko irizpidea (handiena irabazle)
λ (lambda)	Puntuazioa "leuntzen" du, hostoetan hondakin gutxi dagoenean gehiegi fidatu ez dadin
γ (gamma)	Irabazi txikiko zatiketak automatikoki kentzen ditu (inaustea) $\rightarrow$ gainegokitzea saihesten du
η (eta)	Urrats txikiak ematen ditu eguneratzean, Gradient Boosting-eko v bezala

Gradient Boosting arrunta eta XGBoost **ideia berean** oinarritzen dira (hondakinak segidan zuzentzea), baina XGBoost-ek formula propioak eta inaustea erabiltzen ditu azkarrago eta sendoago izateko, batez ere datu-multzo handi eta zaratatsuetan.